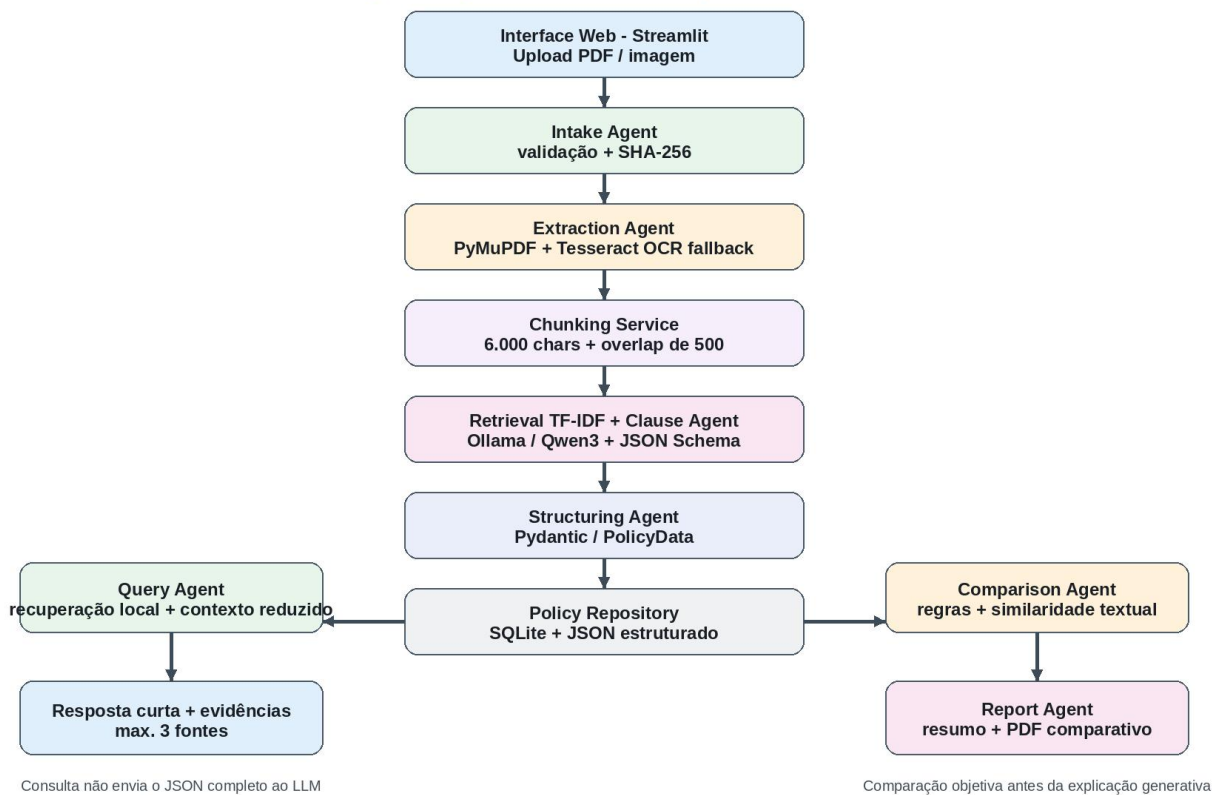


POLICYCOMPARE AI

Plataforma Inteligente para Análise, Consulta e Comparação de Apólices

RELATÓRIO TÉCNICO

PolicyCompare AI v1.2 - Arquitetura Atual



MVP local com IA Generativa, OCR, agentes especializados e rastreabilidade por evidência

Versão: 1.2 | Data: 06 de outubro de 2026

Sumário executivo do documento

Seção	Conteúdo
1	Resumo executivo e objetivos
2	Escopo funcional e requisitos atendidos
3	Arquitetura da solução atualizada
4	Tecnologias utilizadas
5	Agentes e componentes especializados
6	Fluxo completo de processamento
7	Consulta inteligente otimizada - v1.2
8	Modelo de dados, persistência e rastreabilidade
9	Estratégia de comparação
10	Tratamento de erros, configuração e segurança
11	Justificativas arquiteturais
12	Testes e validação
13	Limitações conhecidas
14	Possibilidades de evolução futura
15	Conclusão

1. Resumo executivo

O PolicyCompare AI é uma plataforma de demonstração para automatizar etapas de leitura, extração, consulta e comparação de apólices de seguro. A solução foi projetada para documentos longos e heterogêneos, incluindo PDFs com camada textual e documentos digitalizados que exigem OCR. A arquitetura combina processamento determinístico, IA Generativa local e agentes especializados, mantendo a origem das informações por página e evidência textual.

Na versão atualizada v1.2, o principal aperfeiçoamento está no fluxo de consulta. O Query Agent deixou de enviar todo o JSON da apólice ao modelo de linguagem. Agora, o sistema realiza recuperação local sobre os dados estruturados, seleciona um conjunto pequeno de itens relevantes e envia apenas esse contexto ao LLM. A mudança reduz latência, uso de memória e ocorrências de timeout em computadores locais, além de manter as evidências sob controle da aplicação.

Princípio arquitetural: IA interpreta o conteúdo; código valida, persiste, seleciona evidências e executa comparações objetivas sempre que possível.

1.1 Objetivos da solução

- Receber apólices em PDF e formatos de imagem.
- Extrair automaticamente texto, aplicando OCR somente quando necessário.
- Identificar dados gerais, coberturas, franquias, limites, exclusões e cláusulas especiais.
- Estruturar a informação em um modelo canônico validado.
- Permitir consulta em linguagem natural sobre uma apólice processada.

- Comparar duas apólices e classificar diferenças objetivas.
- Apresentar evidências, páginas e um relatório comparativo em PDF.
- Executar localmente sem dependência obrigatória de APIs comerciais externas.

2. Escopo funcional e requisitos atendidos

Requisito	Implementação atual	Status
Leitura de PDF e imagem	Upload de PDF, PNG, JPG/JPEG e TIFF/TIF.	Atendido
Extração automática	PyMuPDF para texto nativo e Tesseract para OCR.	Atendido
IA Generativa	Ollama com modelo local configurável, atualmente qwen3:4b.	Atendido
Dados estruturados	Pydantic / PolicyData com JSON persistido.	Atendido
Consulta das informações	Query Agent com contexto reduzido e evidências.	Atendido
Comparação de apólices	Comparison Agent com regras e similaridade textual.	Atendido
Diferenças ao usuário	Tabela, classificação, evidências e resumo executivo.	Atendido
Interface demonstrável	Aplicação Streamlit com cinco áreas funcionais.	Atendido
Relatório comparativo	PDF gerado pelo Report Agent / ReportLab.	Atendido
Arquitetura multiagente	Sete agentes especializados e workflow LangGraph.	Atendido

3. Arquitetura da solução atualizada

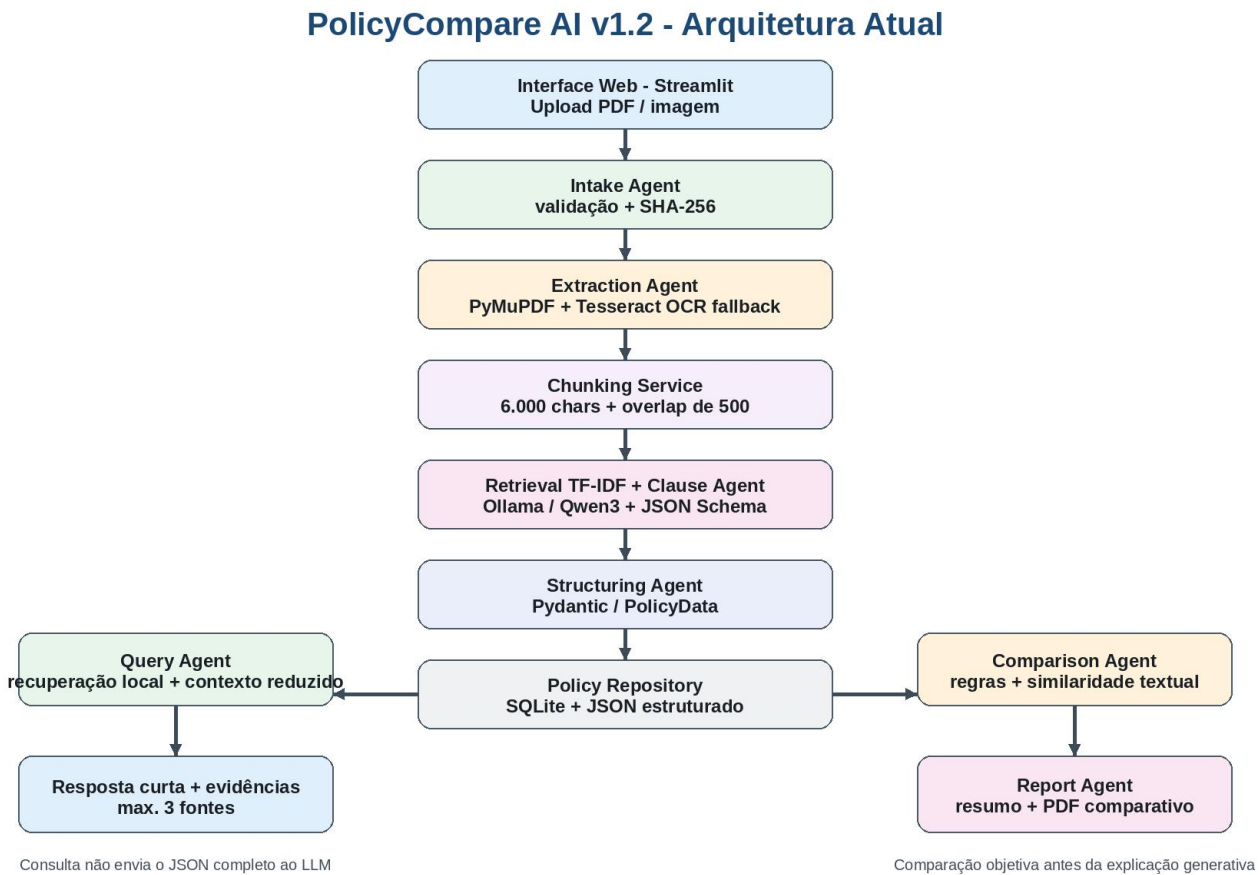


Figura 1 - Arquitetura lógica do PolicyCompare AI v1.2.

3.1 Visão em camadas

Camada	Componentes	Responsabilidade
Interface	Streamlit	Upload, acompanhamento do processamento, consulta, comparação e download de relatórios.
Orquestração	LangGraph / PolicyWorkflow	Controlar o estado e a sequência do pipeline principal de processamento.
Agentes	Intake, Extraction, Clause, Structuring, Query, Comparison, Report	Executar tarefas especializadas e desacopladas.
Serviços	PDF, OCR, Chunking, Retrieval, LLM	Encapsular capacidades técnicas reutilizáveis.
Modelos	Pydantic	Definir contratos estruturados e validações.
Persistência	SQLite	Armazenar apólices e comparações em formato estruturado.
IA local	Ollama / qwen3:4b	Interpretação semântica, resposta de consulta e resumo comparativo.

3.2 Workflow principal

O pipeline de ingestão e processamento é orquestrado por um StateGraph do LangGraph. O estado trafega entre os nodes intake, extract, chunk, analyze, structure e persist. Consulta e comparação são operações posteriores sobre PolicyData já persistido, evitando repetir OCR e extração semântica a cada pergunta.

START -> intake -> extract -> chunk -> analyze -> structure -> persist -> END

4. Tecnologias utilizadas

Tecnologia	Papel na solução	Motivação
Python 3.11+	Linguagem principal	Ecossistema amplo para IA, OCR, documentos, persistência e web.
Streamlit	Interface Web	Entrega rápida de interface funcional para demonstração.
LangGraph	Orquestração do pipeline	Permite representar workflow por estado e evoluir para retries/branching.
Ollama	Servidor de LLM local	Execução local, sem chave externa e com modelo substituível.
Qwen3 4B	Modelo generativo sugerido	Modelo compacto adequado a demonstração local; configurável por .env.
PyMuPDF / fitz	Leitura e renderização de PDF	Extração de texto nativo e renderização de páginas para OCR.
Tesseract OCR	OCR gratuito	Fallback para documentos escaneados e imagens.
Pillow	Manipulação de imagens	Conversão e preparação de imagens para OCR.
Pydantic	Schemas e validação	Contrato explícito entre LLM, agentes e persistência.
scikit-learn	TF-IDF / similaridade	Recuperação leve de chunks na fase de extração.
SQLite	Banco local	Baixa complexidade operacional para MVP.
ReportLab	Geração de PDF	Cria relatório comparativo sem dependência de serviço externo.

5. Agentes e componentes especializados

5.1 Intake Agent

Responsável por validar o arquivo recebido antes de qualquer processamento oneroso.

- Aceita .pdf, .png, .jpg, .jpeg, .tif e .tiff.
- Rejeita arquivo ausente ou vazio.
- Gera identificador determinístico a partir do SHA-256 do conteúdo, utilizando os primeiros 16 caracteres do hash.
- Permite reconhecer novamente o mesmo documento sem depender do nome do arquivo.

5.2 Extraction Agent

Executa a extração física do conteúdo do documento, preservando a página de origem.

- PDF: extração inicial por PyMuPDF.
- Página com menos de 80 caracteres: renderização da página e tentativa de OCR.
- OCR: Tesseract com idioma configurável, por padrão por.
- Imagem: OCR direto e criação de uma página lógica única.
- O método efetivamente utilizado fica identificado no modelo de página.

5.3 Clause Agent

Interpreta semanticamente os trechos mais relevantes da apólice e produz dados estruturados.

- Recebe chunks do documento, não o arquivo bruto.
- Usa RetrievalService com TF-IDF e cosine similarity para selecionar top_k=8 chunks por padrão.
- Solicita ao LLM dados gerais, coberturas, exclusões, cláusulas especiais e observações.
- Exige evidência textual curta, página e confidence de 0 a 1.
- A resposta é validada pelo schema ClauseExtraction.

5.4 Structuring Agent

Converte a saída do Clause Agent para o modelo canônico PolicyData.

- Isola o formato persistido da representação intermediária do LLM.
- Centraliza tipos, listas e campos opcionais.
- Entrega um objeto Pydantic pronto para persistência, consulta e comparação.

5.5 Query Agent - atualizado v1.2

Responde perguntas sobre uma apólice já estruturada sem reenviar todo o documento ou todo o JSON ao modelo.

- Cria candidatos a partir de coberturas, exclusões, cláusulas e observações existentes em PolicyData.
- Normaliza e tokeniza pergunta e candidatos; remove stopwords e calcula sobreposição lexical.
- Seleciona até seis itens relevantes antes da chamada ao LLM.
- Envia dados gerais mais somente os itens ranqueados.
- Usa num_ctx=4096 e num_predict=512 na consulta.
- O LLM retorna apenas answer e inconclusive; as evidências são anexadas deterministicamente pelo código.
- Retorna no máximo três evidências, reduzindo cópia de texto e risco de alucinação de fonte.

5.6 Comparison Agent

Compara duas apólices estruturadas priorizando lógica determinística.

- Pareia nomes de coberturas e exclusões usando SequenceMatcher.
- Compara limites e franquias para classificar condições mais favoráveis quando os valores existem.
- Marca itens existentes somente em A ou somente em B.
- Compara descrições de exclusões por similaridade e usa limiar para equivalência.
- Usa o LLM apenas para produzir resumo executivo; há fallback determinístico se o LLM falhar.

5.7 Report Agent

Transforma a comparação estruturada em artefato compartilhável.

- Gera PDF com identificação das duas apólices.
- Inclui resumo executivo e tabela de diferenças.
- Adiciona aviso de que o resultado deve ser validado por especialista.

6. Fluxo completo de processamento

1. O usuário envia uma ou mais apólices pela aba Processar da interface Streamlit.
2. O arquivo é gravado no diretório de uploads configurado.
3. O Intake Agent valida extensão, existência e tamanho e gera document_id por hash.
4. O Extraction Agent tenta extrair texto nativo por página usando PyMuPDF.
5. Quando uma página possui menos de 80 caracteres, ela é renderizada e submetida ao Tesseract; o OCR substitui o texto original somente se produzir conteúdo maior.
6. O Chunking Service divide o texto em blocos de até 6.000 caracteres, com overlap de 500 caracteres e identificação da página.
7. O RetrievalService calcula TF-IDF sobre os chunks e recupera os trechos mais relacionados aos conceitos de apólice, cobertura, franquia, limite, exclusão e cláusula.
8. O Clause Agent envia o contexto selecionado ao Ollama solicitando saída JSON conforme o schema Pydantic.
9. O LLMService desabilita thinking mode, valida resposta vazia, remove eventuais code fences e valida o JSON pelo Pydantic.
10. O Structuring Agent gera PolicyData e o PolicyRepository persiste o payload JSON no SQLite.
11. A partir deste ponto, consultas e comparações utilizam somente a representação estruturada persistida.

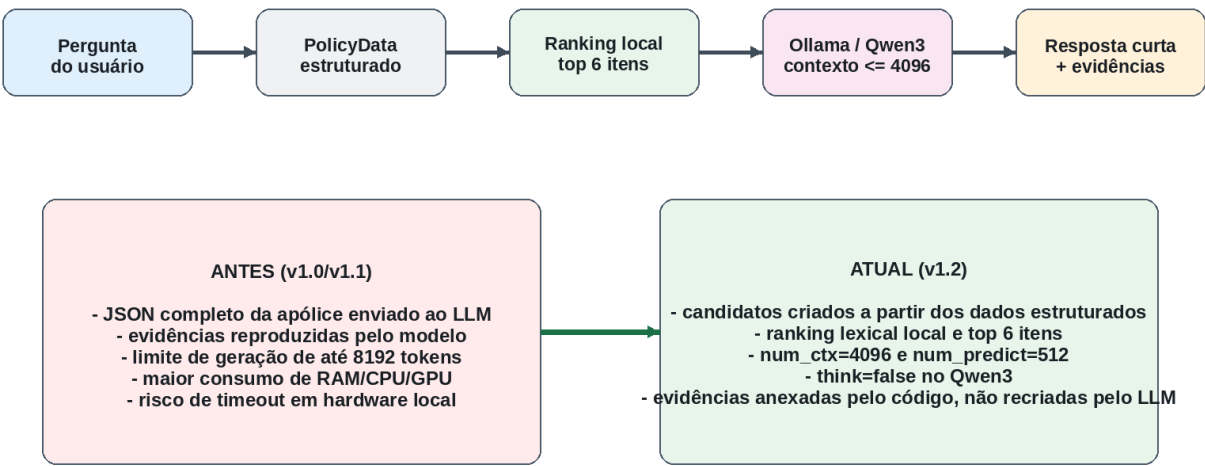
6.1 Parâmetros relevantes do pipeline

Parametro	Valor padrão atual	Uso
OLLAMA_MODEL	qwen3:4b	Modelo local para tarefas generativas.
OCR_LANGUAGE	por	Idioma usado pelo Tesseract.
max_chars_per_chunk	6000	Tamanho máximo aproximado de cada chunk.
chunk_overlap	500	Sobreposição para evitar perda de contexto entre blocos.
top_k_chunks	8	Quantidade máxima de chunks selecionados para o Clause Agent.
Clause num_predict	8192 (padrão do serviço)	Espaco de saída para extração estruturada potencialmente longa.
Query num_ctx	4096	Janela configurada para a consulta otimizada.
Query num_predict	512	Limite reduzido para respostas curtas.
Query top_k	6	Quantidade máxima de candidatos enviados ao LLM.

Parametro	Valor padrão atual	Uso
Query evidences	3	Quantidade máxima de evidências retornadas ao usuário.

7. Consulta inteligente otimizada - v1.2

Fluxo de Consulta Otimizado - v1.2



Objetivo: menor latência, maior previsibilidade e rastreabilidade

Figura 2 - Evolução do fluxo de consulta para reduzir latência e timeout.

Durante os testes da solução, o fluxo inicial de consulta apresentava timeout de 300 segundos em algumas perguntas. O diagnóstico mostrou que o Query Agent enviava `policy.model_dump_json()` completo ao `qwen3:4b`. Uma apólice com muitas coberturas, exclusões e evidências gera um prompt grande e exige maior processamento local.

7.1 Correção aplicada

- **Recuperação antes do LLM:** o agente constroi candidatos estruturados e seleciona somente os mais relacionados a pergunta.
- **Contexto compacto:** dados gerais da apólice são combinados com no máximo seis candidatos.
- **Resposta curta:** o schema gerado pelo LLM contem apenas `answer` e `inconclusive`.
- **Evidência controlada:** página e trecho são reaproveitados dos objetos estruturados; o LLM não precisa copiar nem inventar a fonte.
- **Limites menores:** `num_ctx=4096` e `num_predict=512` para diferenciar consulta de extração.
- **Thinking desabilitado:** `think=false` evita que Qwen3 consuma geração de raciocínio separada quando o objetivo é JSON estruturado.

Benefício: a consulta fica proporcional a informação relevante para a pergunta, e não ao tamanho total da apólice processada.

8. Modelo de dados, persistência e rastreabilidade

A representação canônica é o elemento central da arquitetura. Ela separa a interpretação semântica da forma como os dados são consultados e comparados. Toda informação relevante pode carregar Evidence, composta por página, texto e confidence.

Modelo	Campos principais	Finalidade
PolicyData	policy_id, document_name, insurer, policy_number, insured_name, product, vigências	Cabecalho canônico da apólice.
Coverage	name, description, limit_value, currency, deductible_value, deductible_text, evidence	Representar coberturas e seus parâmetros.
Exclusion	category, description, evidence	Representar riscos ou situações excluídas.
SpecialClause	category, title, description, evidence	Registrar cláusulas relevantes fora do modelo de cobertura/exclusão.
Evidence	page, text, confidence	Rastrear a origem da conclusão.
ComparisonItem	dimension, item, policy_a, policy_b, status, evidence_a, evidence_b	Registrar diferença atômica entre duas apólices.
PolicyComparison	policy_a_id, policy_b_id, summary, items	Agrupar o resultado completo da comparação.

8.1 Persistência SQLite

O repositório cria duas tabelas: policies e comparisons. Em policies, alguns campos principais são mantidos em colunas para identificação e o objeto completo é persistido em payload_json. Essa abordagem simplifica o MVP e permite reconstruir PolicyData com validação Pydantic ao ler o banco.

```
policies(policy_id PK, document_name, insurer, policy_number, product, payload_json, created_at)
comparisons(id PK, policy_a_id, policy_b_id, payload_json, created_at)
```

9. Estratégia de comparação

A comparação foi deliberadamente projetada como híbrida. Valores numéricos e existência de itens são tratados por código; similaridade de texto apoia o pareamento; o LLM é utilizado para redigir um resumo executivo sobre o resultado já calculado.

Classificação	Interpretação
EQUIVALENTE	Não foi identificada diferença material no critério comparado.
MAIS_FAVORAVEL_A	O critério objetivo favorece a apólice A.
MAIS_FAVORAVEL_B	O critério objetivo favorece a apólice B.
SOMENTE_A	O item foi identificado apenas na apólice A.
SOMENTE_B	O item foi identificado apenas na apólice B.
DIFERENTE	Ambas possuem o item, mas o conteúdo ou condição difere.
INCONCLUSIVO	Não há informação suficiente para uma conclusão confiável.

9.1 Regras atualmente implementadas

- Coberturas são pareadas pela melhor similaridade de nome, respeitando itens já utilizados.

- Itens sem correspondência são marcados SOMENTE_A ou SOMENTE_B.
- Quando existem limites e/ou franquias, o agente avalia os valores para classificar favorabilidade.
- Exclusões são pareadas por categoria e suas descrições são comparadas por SequenceMatcher; similaridade igual ou superior a 0,78 e tratada como equivalente.
- Dados gerais como seguradora, produto e vigência são comparados diretamente.
- O resumo gerado pelo LLM recebe uma lista compacta dos resultados; se ocorrer falha, o sistema gera um resumo determinístico com contagem de diferenças.

10. Tratamento de erros, configuração e segurança

10.1 Tratamento de erros

Cenário	Tratamento atual
Formato não suportado	Interrompe no Intake Agent com mensagem de formato.
Arquivo vazio/inexistente	Interrompe antes de OCR/LLM.
Ollama indisponível	Interface sinaliza indisponibilidade antes de processar/consultar.
Timeout HTTP	LLMService converte requests exception em LLMServiceError; consulta exibe mensagem específica para Read timed out.
response vazio do Ollama	LLMService detecta antes do Pydantic e informa modelo, done_reason e tamanho de thinking.
JSON inválido	Tentativa de validação direta e via json.loads; falha gera erro com preview da resposta.
Thinking do Qwen3	Desabilitado explicitamente com think=false nas chamadas estruturadas e textuais.
Falha no resumo comparativo	Comparison Agent aplica fallback determinístico.

10.2 Configuração

- Configuração por .env usando pydantic-settings.
- URL padrão do Ollama: http://localhost:11434.
- Modelo configurável sem alterar o código.
- Diretórios de banco, upload e processamento criados automaticamente.
- Arquivo .env real deve permanecer fora do controle de versão.

10.3 Segurança do MVP

- O processamento pode ocorrer inteiramente localmente, evitando envio obrigatório da apólice a provedores externos.
- Não há chaves de API comerciais obrigatórias na configuração padrão.
- O MVP ainda não implementa autenticação, autorização por perfil, criptografia em repouso, segregação por usuário, políticas de retenção ou trilha de auditoria completa.
- Apólices reais podem conter dados pessoais e informações comerciais sensíveis; esses controles seriam obrigatórios em uma implantação corporativa.

11. Justificativas das decisões arquiteturais

Decisão	Justificativa	Trade-off
LLM local via Ollama	Facilita demonstração offline e reduz exposição de documentos a serviços externos.	Qualidade e velocidade dependem do hardware local.
qwen3:4b como sugestão	Modelo relativamente compacto e suficiente para MVP; pode ser	Modelos pequenos podem perder

Decisão	Justificativa	Trade-off
	substituído.	nuances jurídicas.
think=false	Evita resposta final vazia e processamento desnecessário em tarefas estruturadas.	Não explora raciocínio estendido do modelo.
PyMuPDF + OCR fallback	PDF textual e processado rapidamente; OCR é usado apenas quando necessário.	Layouts complexos podem exigir OCR/document intelligence especializado.
TF-IDF no Clause Agent	Sem banco vetorial e sem modelo adicional de embeddings.	Recuperação lexical tem menor capacidade semântica.
Ranking local no Query Agent	Reduz contexto e timeout e reutiliza dados já estruturados.	Perguntas com sinônimos muito distantes podem recuperar poucos candidatos.
Pydantic / JSON Schema	Evita parsing livre e estabelece contrato de dados.	Schemas muito rígidos podem exigir evolução quando surgirem novos ramos.
SQLite	Instalação mínima e suficiente para demonstração.	Não é o banco recomendado para grande escala/multiusuário.
Comparação determinística	Torna a classificação auditável e testável.	Regras precisam evoluir por produto/ramo de seguro.

12. Testes e validação

A versão atual inclui testes unitários para comportamentos deterministas e foi novamente validada durante a preparação deste relatório.

Verificação	Resultado
pytest -q	2 testes aprovados em 0,20 s no ambiente de validação.
Intake determinístico	Mesmo arquivo gera o mesmo identificador por hash.
Comparação objetiva	Teste cobre classificação de limite mais favorável.
Validação Pydantic	Objetos persistidos e retornos de LLM são submetidos aos schemas.
PDFs de demonstração	O projeto inclui duas apólices sintéticas para roteiro de demonstração.

```
pytest -q
..
2 passed in 0.20s [100%]
```

12.1 Roteiro recomendado para demonstração

12. Iniciar o Ollama e confirmar o modelo configurado.
13. Executar a aplicação Streamlit.
14. Criar ou selecionar os PDFs de demonstração.
15. Processar duas apólices e conferir o JSON estruturado.
16. Na aba Apólices, revisar quantidades de coberturas, exclusões e cláusulas.
17. Na aba Consultar, fazer perguntas objetivas como: Qual é a franquia de danos elétricos? ou Existe exclusão para guerra?
18. Confirmar que a resposta apresenta evidências e páginas quando disponíveis.
19. Na aba Comparar, selecionar A e B e executar a comparação.
20. Revisar classificações, evidências e baixar o PDF comparativo.

13. Limitações conhecidas

- O MVP não substitui análise jurídica, atuarial ou técnica de um especialista em seguros.
- A taxonomia de coberturas, exclusões e cláusulas é genérica e pode variar significativamente por ramo.
- O Clause Agent recupera top 8 chunks por uma consulta lexical fixa; informações muito dispersas no documento podem ficar fora do contexto selecionado.
- Tabelas complexas, documentos em múltiplas colunas, carimbos, assinaturas e imagens de baixa qualidade podem reduzir a qualidade da extração.
- Tesseract é uma solução OCR gratuita e eficaz para o MVP, mas não realiza compreensão avançada de layout.
- O ranking atual do Query Agent é lexical e pode ter dificuldade com sinônimos, abreviações ou perguntas conceitualmente distantes do texto estruturado.
- O desempenho do qwen3:4b varia conforme CPU, RAM, GPU e tamanho do contexto.
- SQLite não foi projetado para alta concorrência ou operação distribuída.
- Não há autenticação, RBAC, criptografia, versionamento de documentos, trilha completa de auditoria ou políticas de retenção.
- A comparação atual trabalha com duas apólices por vez e cobre um conjunto inicial de dimensões.

14. Possibilidades de evolução futura

Evolução	Descrição	Benefício esperado
RAG vetorial	Substituir/combinar TF-IDF e ranking lexical com embeddings multilíngues e Qdrant, pgvector ou Chroma.	Melhor recuperação semântica e sinônimos.
OCR de layout	Integrar modelos de document understanding/layout para tabelas e formulários.	Melhor extração de PDFs complexos.
Ontologias por ramo	Criar schemas e vocabulários para Cyber, D&O, Auto, Patrimonial, Vida etc.	Comparações mais precisas e específicas.
Human-in-the-loop	Fila de revisão para baixa confiança, conflitos e itens inconclusivos.	Governança e qualidade.
Avaliação automatizada	Dataset rotulado e métricas de precision, recall, F1 e acurácia por campo.	Medição objetiva de qualidade.
Observabilidade LLM	Registrar latência, contexto, modelo, versão de prompt, falhas e consumo.	Diagnóstico e operação em produção.
Processamento paralelo	Processar páginas/seções em paralelo e consolidar resultados.	Menor tempo em documentos longos.
Banco corporativo	Migrar persistência para PostgreSQL/pgvector e storage de objetos.	Escala, concorrência e governança.
API FastAPI	Expor upload, consulta e comparação como serviços REST.	Integração com portais e sistemas corporativos.
Segurança empresarial	SSO/OIDC, RBAC, criptografia, auditoria, masking e retenção.	Uso seguro com documentos reais.
Comparação N apólices	Expandir a matriz de comparação para tres ou mais documentos.	Apoio a processos de cotação e concorrência.
Evidência navegável	Abrir a página do PDF original e destacar o trecho citado.	Auditoria e experiência do usuário.
Exportações	Excel/CSV/JSON e relatório executivo mais rico.	Interoperabilidade e análise externa.

14.1 Arquitetura alvo sugerida para produção

- Frontend corporativo ou portal integrado.
- API FastAPI e fila de processamento assíncrono.
- Object storage para documentos e versões.

- PostgreSQL para dados estruturados e pgvector/Qdrant para embeddings.
- OCR/document intelligence especializado.
- Orquestração com LangGraph incluindo retries, checkpoints e human-in-the-loop.
- Gateway de modelos permitindo LLM local ou corporativo homologado.
- Observabilidade, avaliação, versionamento de prompts e trilha de auditoria.

15. Estrutura do repositório

```
insurance-policy-ai/
|-- app/
|   |-- agents/
|   |   |-- intake_agent.py
|   |   |-- extraction_agent.py
|   |   |-- clause_agent.py
|   |   |-- structuring_agent.py
|   |   |-- query_agent.py
|   |   |-- comparison_agent.py
|   |   `-- report_agent.py
|   |-- config/
|   |-- models/
|   |-- repositories/
|   |-- services/
|   `-- workflows/
|-- frontend/streamlit_app.py
|-- tests/
|-- samples/
|-- docs/
|-- requirements.txt
|-- docker-compose.yml
|-- .env.example
|-- run.py
`-- README.md
```

Apêndice A - Exemplo do modelo estruturado

```
{
  "policy_id": "doc-<hash>",
  "document_name": "apolice.pdf",
  "insurer": "Seguradora Exemplo",
  "policy_number": "123456",
  "effective_start": "2026-01-01",
  "effective_end": "2026-12-31",
  "coverages": [
    {
      "name": "Danos eletricos",
      "limit_value": 200000,
      "currency": "BRL",
      "deductible_value": 5000,
      "evidence": {
        "page": 8,
        "text": "Trecho de origem da informacao...",
        "confidence": 0.95
      }
    }
  ]
}
```

Apêndice B - Principais configurações do .env

```
OLLAMA_BASE_URL=http://localhost:11434  
OLLAMA_MODEL=qwen3:4b  
LLM_MODE=ollama  
OCR_LANGUAGE=por  
DATABASE_PATH=data/policies.db  
UPLOAD_DIR=data/uploads  
PROCESSED_DIR=data/processed
```